SIX MONTHS INDUSTRIAL TRAINING PROJECT REPORT

ON

"AI CODE REVIEW & QUALITY ANALYZER"

SUBMITTED IN FULFILLMENT OF THE DEGREE OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

B.Tech

(Computer Science & Engineering with Specialization in Artificial Intelligence & Machine Learning)

SUBMITTED BY

Aryan

322021013

(B. Tech CSE 8th Semester)

SUBMITTED TO

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SRI SAI UNIVERSITY, PALAMPUR
JUNE 2026

CERTIFICATION (BY COMPANY)

ThinkNEXT Technologies Private Limited

ThinkNEXT™
Innovation at every step...
ISO 9001:2015 Certified Company

Scan and verify your Certificate
Certificate ID:832843
Ref.No. TNT/C-26/18820

Iconic Business Summit AWARDS WINNER 2021
Nation's Business Pride AWARDS WINNER 2021
National Gratitude AWARDS WINNER 2020
Asia's Quality & Entrepreneurship AWARDS WINNER 2019
Business Leaders AWARDS WINNER 2019
NATIONAL ICON AWARDS WINNER 2018

Certificate

Provisional

This Certificate do hereby recognizes that

Aryan S/o Ranjeet Singh

has successfully completed Industrial Training Program from

20th January 2026 to *04th June 2026*

in *AI & ML* Grade *A*

Certificate of Excellence

ISO Certified

Member of Confederation of Indian Industry

ISTQB International Software Testing Qualifications Board Affiliated

For ThinkNEXT Technologies Pvt. Ltd.
Authorised Signatory
Member Training Division

ThinkNEXT Technologies Pvt. Ltd.
Director
Director

Corporate Office: S.C.F 113, Phase XI, Mohali (Punjab)

A Outstanding 100-90% B Excellent 89-80% C Very Good 79-70% D Good 69-60% E Satisfactory 59-50%

DECLARATION

I hereby certify that the work which is being presented in the project entitled "**AI Code Review & Quality Analyser**" in fulfilment of requirements for the award of degree of B.Tech (CSE) submitted in the Department of Computer Science & Engineering at **SRI SAI UNIVERSITY PALAMPUR** is an authentic record of my own work carried out during a period from January 2026 to June 2026 under the supervision of Assoc. Software Engineer "**Balwinder Kumar**". The matter presented in this project has not been submitted by me in any other University / Institute for the award of B.Tech Degree.

Name: Aryan
Roll No.: 322021013
B. Tech CSE 8th Semester

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

Guide Name: Balwinder Kumar
(Assoc. Software Engineer)

Asst. Prof Er. Yamini Sood
(Head of Department)

ABSTRACT

The rapid growth of software development has increased the need for efficient and reliable code quality assessment techniques. Traditional manual code reviews are often time-consuming, inconsistent, and difficult to perform for large-scale projects. To address these challenges, the AI Code Review & Quality Analyzer has been developed as an intelligent system that automates code quality evaluation using Machine Learning and Artificial Intelligence techniques.

The proposed system analyzes Python source code and GitHub repositories by extracting important software metrics such as Lines of Code (LOC), Source Lines of Code (SLOC), Logical Lines of Code (LLOC), Cyclomatic Complexity, Maintainability Index, Function Count, and Class Count. These metrics are processed using a trained XGBoost Machine Learning model to predict the quality of the code as Good, Average, or Poor.

The system also supports GitHub repository analysis by automatically fetching Python (.py) and Jupyter Notebook (.ipynb) files from public repositories. It performs file-level analysis, calculates an overall Repository Health Score, and provides detailed insights into software quality. Furthermore, the integration of a Large Language Model (LLM) generates human-readable explanations and recommendations to help developers improve code maintainability and readability.

A user-friendly web interface has been developed using Streamlit, allowing users to analyze code, visualize metrics through interactive charts, and download professional PDF reports. The generated reports include quality predictions, repository metrics, health scores, file-wise analysis, and AI-generated reviews.

Keywords: Artificial Intelligence, Machine Learning, Code Review, Software Quality Analysis, GitHub Repository Analysis, XGBoost, Streamlit, Repository Health Score.

COMPANY PROFILE

ThinkNEXT Technologies Private Limited is an ISO 9001:2015 certified Information Technology and Digital Marketing company headquartered in Mohali (Chandigarh), Punjab, India. Established in 2011, the company provides a wide range of services including Web Design and Development, Digital Marketing, Mobile Application Development, ERP Solutions, Learning Management Systems (LMS), Chatbot Development, Industrial Training, Internships, and Software Development Services. The company is registered under the Ministry of Corporate Affairs, Government of India.

ThinkNEXT has served clients across multiple countries including India, USA, Canada, UK, Australia, New Zealand, and Singapore. The company focuses on delivering innovative technology solutions and digital transformation services for businesses, educational institutions, and startups.

The organization is recognized for its contributions in digital marketing, web development, and industrial training. Over the years, it has received several national-level awards and certifications and has established itself as a reputed training and software development company in North India. The company is also associated with various technology and skill-development initiatives.

ThinkNEXT provides practical industrial training programs in emerging technologies such as Artificial Intelligence, Machine Learning, Data Science, Python, Web Development, Digital Marketing, and Software Testing. These programs are designed to bridge the gap between academic learning and industry requirements by offering hands-on project experience and professional skill development.

The company operates from its corporate office located in Mohali, Punjab, and continues to expand its services in software development, digital marketing, automation solutions, and professional training. Its mission is to deliver quality technology solutions while empowering students and professionals with industry-relevant skills.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Background of The Study

Software has become an integral part of modern society, powering applications across industries such as healthcare, education, finance, transportation, and communication. As software systems continue to grow in complexity and scale, maintaining high-quality source code has become a critical challenge for developers and organizations. The quality of software directly impacts its reliability, maintainability, performance, and security. Poorly written code can result in software failures, increased maintenance costs, reduced developer productivity, and negative user experiences.

Code quality refers to the degree to which source code adheres to established programming standards, best practices, readability requirements, and maintainability principles. High-quality code is easier to understand, modify, test, and extend. Therefore, software organizations invest significant resources in code reviews and quality assurance processes to ensure that software products meet desired quality standards.

Code review is one of the most widely adopted practices in software engineering. It involves systematically examining source code to identify bugs, improve readability, enforce coding standards, and ensure maintainability. Traditionally, code reviews are conducted manually by experienced developers. While manual reviews are effective, they require considerable time and effort, especially for large software projects containing thousands of lines of code distributed across multiple files.

With the rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML), new opportunities have emerged for automating software engineering tasks. AI-powered tools can analyze source code, detect patterns, identify quality issues, and generate intelligent recommendations. These technologies have the potential to significantly reduce the workload associated with manual code reviews while improving consistency and accuracy.

The AI Code Review & Quality Analyzer project has been developed to leverage the capabilities of Machine Learning and Artificial Intelligence for automated code quality assessment. The system evaluates Python source code and GitHub repositories using software

engineering metrics and predictive models. It provides developers with detailed insights regarding code quality, maintainability, complexity, and overall repository health. By

combining static code analysis, machine learning prediction, and AI-generated explanations, the proposed system aims to improve software quality and enhance developer productivity.

1.2 Introduction to the Review

Code review is a systematic process of examining source code to identify defects, improve design quality, enforce coding standards, and ensure maintainability. It is considered one of the most important quality assurance activities in software development. The primary goal of code review is to detect issues before software is deployed, thereby reducing the likelihood of bugs and improving overall software reliability.

Code reviews are typically performed by peers or senior developers who analyze source code changes and provide feedback to the author. During the review process, reviewers evaluate several aspects of the code, including readability, logic, performance, security, documentation, and adherence to coding standards.

The benefits of code review include:

- Early detection of software defects.
- Improved code readability and maintainability.
- Knowledge sharing among team members.
- Consistent coding standards across projects.
- Reduced software maintenance costs.
- Enhanced software reliability and performance.

Despite these benefits, traditional code review methods face several challenges. Manual reviews can be time-consuming, especially for large repositories. The effectiveness of reviews often depends on the experience and expertise of individual reviewers. Additionally, human reviewers may overlook certain issues due to fatigue, time constraints, or lack of domain knowledge.

As software projects continue to expand in size and complexity, organizations increasingly seek automated solutions that can complement or partially replace manual review processes. Intelligent code analysis systems powered by Artificial Intelligence offer a promising approach for addressing these challenges.

1.3 Need for Automation

The increasing complexity of modern software systems has created a growing demand for automated code quality assessment tools. Large software repositories often contain hundreds of files and thousands of lines of code, making comprehensive manual reviews difficult and resource - intensive.

Several factors contribute to the need for automation:

Large Software Repositories

Modern software projects frequently involve multiple developers working on extensive codebases. Reviewing every file manually becomes impractical as project size increases.

Time Consuming Review Process

Manual code reviews require developers to spend significant amounts of time analyzing source code. This can delay software development and reduce productivity.

Human Error Inconsistencies

Human reviewers may overlook defects or evaluate code differently based on their experience and personal preferences. This can lead to inconsistent quality assessments.

Need for Continuous Quality Monitoring

Agile and DevOps practices emphasize rapid software delivery and continuous integration. Organizations require automated tools that can provide instant feedback regarding code quality.

Increasing Adoption of AI technologies

Artificial Intelligence and Machine Learning technologies have demonstrated significant potential in automating complex analytical tasks. These technologies can analyze code patterns, identify quality issues, and generate recommendations more efficiently than traditional methods .

To address these challenges, an intelligent automated system capable of evaluating source code and repositories is required. The proposed AI Code Review & Quality Analyzer fulfills this requirement by combining software metrics, machine learning algorithms, and AI-generated explanations.

1.4 Overview of the Proposed System

The AI Code Review & Quality Analyzer is an intelligent web-based application designed to automate the process of code quality assessment. The system accepts either Python source code or a GitHub repository URL as input and performs a comprehensive analysis using software engineering metrics and machine learning techniques.

The workflow of the proposed system consists of the following stages:

1. Source code collection.
2. Feature extraction.
3. Quality prediction.
4. Repository analysis.
5. Health score generation.
6. AI explanation generation.
7. Report generation.

The system extracts various software metrics such as:

- Lines of Code (LOC)
- Source Lines of Code (SLOC)
- Logical Lines of Code (LLOC)
- Cyclomatic Complexity
- Maintainability Index
- Function Count
- Class Count
- Comment Density

These metrics are provided to a trained XGBoost machine learning model that predicts the quality of the code as Good, Average, or Poor.

In addition to single-file analysis, the system supports GitHub repository analysis. It automatically retrieves Python (.py) files and Jupyter Notebook (.ipynb) files from public repositories and evaluates each file individually.

A Repository Health Score is calculated using maintainability, complexity, documentation quality, and code structure metrics. The application also integrates a Large Language Model (LLM) to generate natural language explanations and recommendations.

The final output includes interactive visualizations, file-level analysis, repository health assessments, and downloadable PDF reports.

1.5 Problem Statement

Software developers often struggle to maintain code quality across large and complex projects. Traditional manual code review processes are time-consuming, subjective, and difficult to scale. Existing static analysis tools provide software metrics but often fail to offer intelligent interpretations and actionable recommendations.

Furthermore, analyzing large GitHub repositories manually can be challenging due to the number of files involved and the diversity of code structures. Developers require a system that can automatically evaluate code quality, identify maintainability issues, generate repository-level insights, and provide meaningful recommendations.

Therefore, there is a need for an intelligent and automated solution capable of analyzing source code and GitHub repositories, predicting code quality, generating repository health scores, and producing AI-powered recommendations.

1.6 Objective

The primary objective of this project is to develop an AI-powered platform capable of automating code quality assessment and repository analysis.

The specific objectives are:

- To analyze Python source code automatically.
- To extract software engineering metrics.
- To predict code quality using Machine Learning.
- To perform GitHub repository analysis.
- To provide file-level quality assessments.
- To generate repository health scores.
- To create AI-powered explanations and recommendations.
- To provide interactive visualizations.
- To generate professional PDF report.

1.7 Scope of Project

The scope of the AI Code Review & Quality Analyzer is focused on evaluating Python source code and GitHub repositories. The system supports both individual code analysis and repository-level assessment.

The project includes:

- Python source code analysis.
- GitHub repository analysis.
- Software metric extraction.
- Machine learning-based quality prediction.
- Repository health scoring.
- AI-generated code review explanations.
- Visualization of code metrics.
- PDF report generation.

The current implementation is limited to Python-based projects and public GitHub repositories. Future enhancements may extend support to additional programming languages and advanced software quality assessment techniques.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Introduction

Requirement Analysis is one of the most important phases of the Software Development Life Cycle (SDLC). It involves identifying, analyzing, documenting, and validating the functional and non-functional requirements of a software system. The primary objective of requirement analysis is to ensure that the developed system satisfies user expectations and fulfills the intended objectives efficiently.

The AI Code Review & Quality Analyzer has been developed to automate software quality assessment using Machine Learning and Artificial Intelligence techniques. The system analyzes Python source code and GitHub repositories to predict code quality, generate repository health scores, and provide AI-powered recommendations. To ensure successful implementation, it is necessary to clearly define the system requirements, constraints, and functionalities.

This chapter discusses the existing system, limitations of current approaches, proposed solution, functional requirements, non-functional requirements, hardware requirements, software requirements, and feasibility analysis of the proposed system.

2.2 Existing System

Code quality assessment is traditionally performed through manual code reviews and static code analysis tools. Software organizations often rely on experienced developers and quality assurance teams to inspect source code and identify defects before deployment.

2.2.1 Manual Code Review

Manual code review is a process in which developers inspect source code line by line to identify errors, maintain coding standards, and improve software quality. It is considered one of the most effective quality assurance practices in software development.

The manual review process generally involves:

1. Reviewing source code changes.
2. Identifying bugs and vulnerabilities.
3. Checking coding standards.

4. Suggesting improvements.
5. Approving or rejecting code changes.

Although manual reviews are useful, they suffer from several drawbacks.

Limitation of Manual Code Review

- Time-consuming for large projects.
- Dependence on reviewer expertise.
- Human errors and oversight.
- Inconsistent evaluation criteria.
- Difficulty in analyzing large repositories.

2.2.2 Static Code Analysis Tools

Several software tools are available for automated code analysis. Popular examples include:

Pylint

Pylint analyzes Python code and identifies syntax errors, coding standard violations, and potential bugs.

SonarQube

SonarQube is an enterprise-level platform used to monitor software quality, maintainability, and security vulnerabilities.

Radon

Radon calculates software metrics such as:

- Cyclomatic Complexity
- Maintainability Index
- Lines of Code

Limitation of Existing Tools

- Lack of Machine Learning-based quality prediction.
- Limited repository-level assessment.
- No repository health scoring mechanism.
- Lack of AI-generated explanations.
- Limited reporting capabilities.
- Difficulty in interpreting raw metrics.

These limitations highlight the need for a more intelligent and automated solution.

2.3 Proposed System

The proposed AI Code Review & Quality Analyzer is an intelligent web-based application designed to automate code quality assessment and repository analysis.

The system integrates software engineering metrics, Machine Learning algorithms, GitHub repository analysis, and Large Language Models to provide comprehensive insights into software quality.

The application performs the following functions:

- Accepts Python code as input.
- Extracts software quality metrics.
- Predicts code quality using XGBoost.
- Analyzes GitHub repositories.
- Performs file-level quality analysis.
- Calculates repository health scores.
- Generates AI-powered recommendations.
- Creates downloadable PDF reports.

The proposed system eliminates many limitations of traditional review methods by providing automated, consistent, and scalable analysis.

Advantages of the Proposed System

The proposed system offers several advantages:

1. Automated code quality assessment.
2. Faster analysis compared to manual reviews.
3. Objective evaluation using Machine Learning.
4. Repository-level analysis capabilities.
5. AI-generated recommendations and explanations.
6. Interactive visualizations and charts.
7. Professional PDF report generation.
8. Improved software maintainability assessment.
9. Enhanced developer productivity.

2.4 Functional Requirements

Functional requirements describe the operations and services that the system must provide.

FR-1: Code Analysis

The system shall allow users to paste Python source code into the application and perform quality analysis.

FR-2: Feature Extraction

The system shall extract software engineering metrics including:

- Lines of Code (LOC)
- Source Lines of Code (SLOC)
- Logical Lines of Code (LLOC)
- Cyclomatic Complexity
- Maintainability Index
- Function Count
- Class Count
- Comment Count

FR-3: Machine Learning Prediction

The system shall utilize a trained XGBoost model to classify code quality into:

- Good
- Average
- Poor

FR-4: GitHub Repository Analysis

The system shall accept public GitHub repository URLs and retrieve Python and Jupyter Notebook files automatically.

FR-5: File-Level Analysis

The system shall analyze each file individually and provide quality predictions.

FR-6: Repository Health Score

The system shall compute an overall health score based on repository metrics.

FR-7: AI Explanation Generation

The system shall generate natural language explanations and recommendations using a Large Language Model.

FR-8: Visualization Module

The system shall display interactive graphs and charts representing extracted metrics.

FR-9: PDF Report Generation

The system shall generate downloadable PDF reports containing analysis results and recommendations.

2.5 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Performance

The application should provide analysis results within a reasonable response time and handle repositories containing multiple files efficiently.

Reliability

The system should provide accurate and consistent outputs under normal operating conditions.

Scalability

The application should be capable of handling larger repositories and additional features in future versions.

Security

The system should securely interact with external APIs and protect user inputs from unauthorized access.

Maintainability

The software should follow modular design principles to facilitate future updates and maintenance.

Usability

The user interface should be intuitive, user-friendly, and easy to navigate.

Portability

The application should operate across different platforms supporting Python and web browsers.

2.6 Hardware Requirements

The hardware requirements for the proposed system are listed below:

Components	Requirement
Processor	Intel Core i3 or Higher
RAM	Minimum 4GB
Storage	1GB Free Space
Internet Connection	Required
Display	Standard Monitor

The system does not require high-end hardware and can operate on standard personal computers.

2.7 Software Requirements

The software requirements for the proposed system are shown below:

Software/Tool	Purpose
Python	Programming Language
Streamlit	User Interface Development
Pandas	Data Processing
Numpy	Numerical Operations
Scikit-Learn	Machine Learning Utilities
XGBoost	Quality Prediction Model
Plotly	Data visualization
Requests	GitHub API Communication
ReportLab	PDF Report Generation
GitHub API	Repository Data Retrieval
Groq/OpenAI API	AI Explanation Generation

The use of open-source tools significantly reduces development costs.

2.8 Feasibility Study

A feasibility study evaluates whether the project can be successfully implemented within available resources and constraints.

2.8.2 Technical Feasibility

The project is technically feasible because all required technologies are readily available.

The system is developed using:

- Python
- Streamlit
- XGBoost
- Pandas
- Plotly
- ReportLab

These technologies are widely adopted, well-documented, and compatible with one another.

Furthermore, GitHub APIs and Large Language Model APIs provide reliable support for repository analysis and AI-generated recommendations.

2.8.2 Economic Feasibility

The project is economically feasible because it primarily utilizes open-source software and freely available development tools.

The development environment requires:

- No specialized hardware.
- No expensive software licenses.
- Minimal operational costs.

Therefore, the project can be implemented with very low investment.

2.8.3 Operational Feasibility

The proposed system is operationally feasible because it is easy to use and requires minimal technical expertise.

Developers can simply:

1. Paste source code.
2. Enter a GitHub repository URL.
3. View analysis results.
4. Download reports.

The user-friendly Streamlit interface ensures smooth interaction and improves user experience.

2.9 Summary

This chapter discussed the requirements and feasibility of the AI Code Review & Quality Analyzer. The limitations of traditional code review methods and static analysis tools were examined, highlighting the need for an intelligent automated solution. The proposed system addresses these challenges through Machine Learning, GitHub repository analysis, AI-powered explanations, and repository health scoring.

The chapter also defined the functional and non-functional requirements, hardware and software specifications, and feasibility considerations necessary for successful implementation. The next chapter presents the detailed system design, including architecture diagrams, data flow diagrams, use case diagrams, activity diagrams, and overall system structure.

CHAPTER 3: SYSTEM DESIGN

3.1 Introduction

System Design is a critical phase in software development that transforms requirements into a structured blueprint for implementation. It defines the architecture, components, modules, interfaces, and data flow within the system. A well-designed system improves maintainability, scalability, reliability, and performance.

The AI Code Review & Quality Analyzer is designed as a modular web application that integrates software metric extraction, Machine Learning prediction, GitHub repository analysis, AI-generated explanations, and PDF report generation. The system follows a layered architecture where each module performs a specific function while interacting with other modules to deliver complete code quality analysis.

The design focuses on simplicity, modularity, and extensibility, allowing future enhancements such as multi-language support, security analysis, and CI/CD integration.

3.2 System Architecture

The architecture of the AI Code Review & Quality Analyzer consists of several interconnected modules that work together to process user input and generate meaningful outputs.

3.2.1 System Architecture Diagram

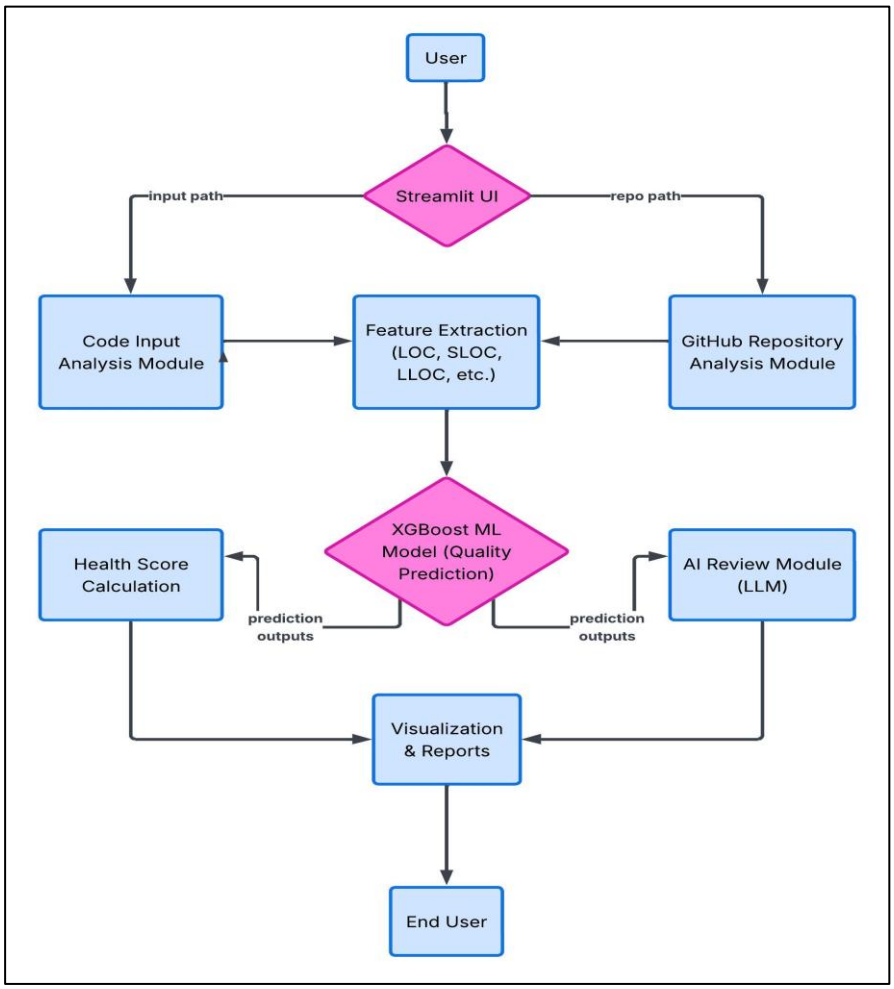


Fig.3.2.1 System Architecture Diagram

Architecture Explanation

The user interacts with the Streamlit-based web interface. The system accepts either Python source code or a GitHub repository URL. The input is processed by the Feature Extraction Module, which calculates software quality metrics. These metrics are passed to the XGBoost Machine Learning model to predict code quality. The prediction results are then used to compute a Repository Health Score and generate AI-powered explanations. Finally, the results are displayed through visualizations and downloadable PDF reports.

3.3 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) illustrates the movement of data between different processes and system components.

3.3.1 DFD Level 0 (Context Diagram)

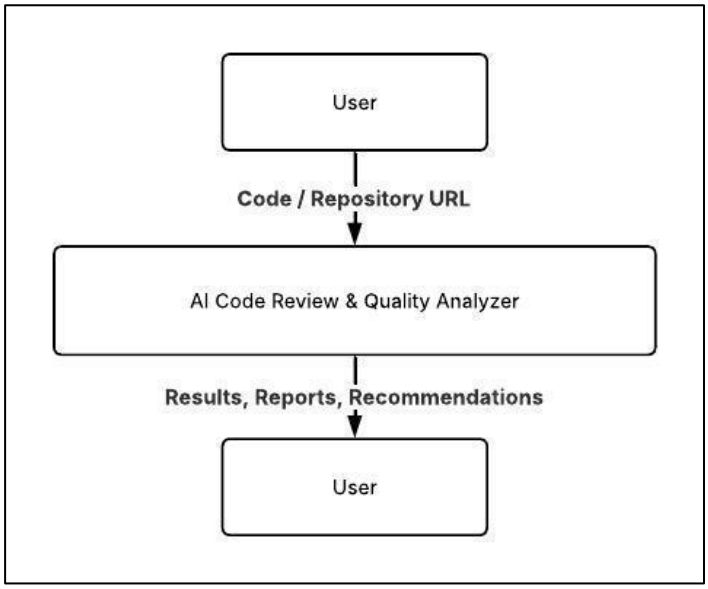


Fig 3.3.1 DFD level 0

Description

The user provides Python code or a GitHub repository URL to the system. The system processes the input and returns code quality predictions, repository health scores, metrics, visualizations, and AI-generated recommendations.

3.3.2 DFD Level 1

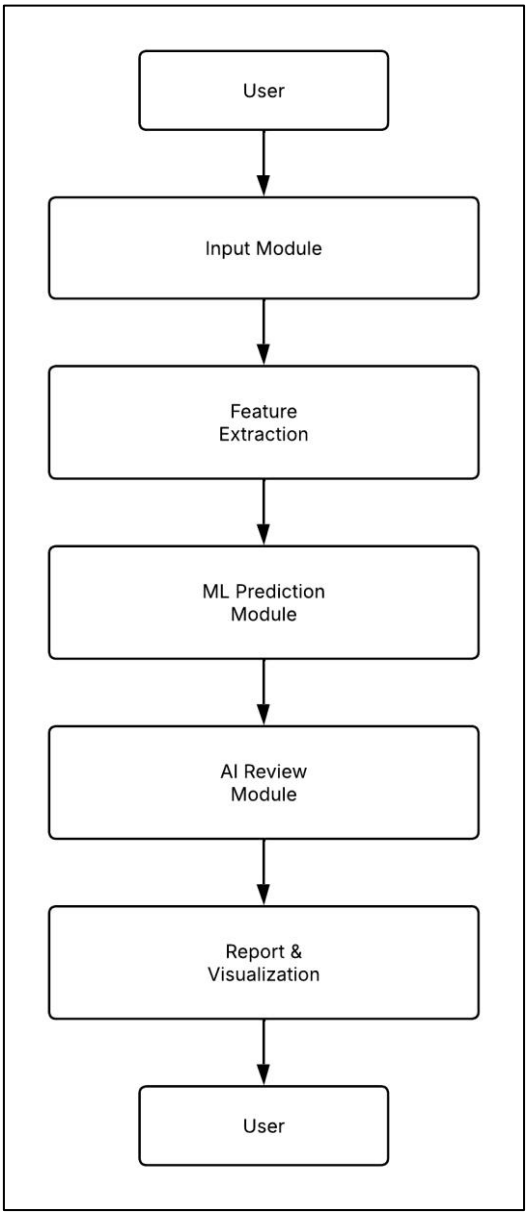


Fig 3.3.2 DFD Level 1

Description

Level 1 DFD decomposes the system into major functional modules. Input data is processed through feature extraction and machine learning prediction before generating reports and recommendations.

3.3.3 DFD Level 2

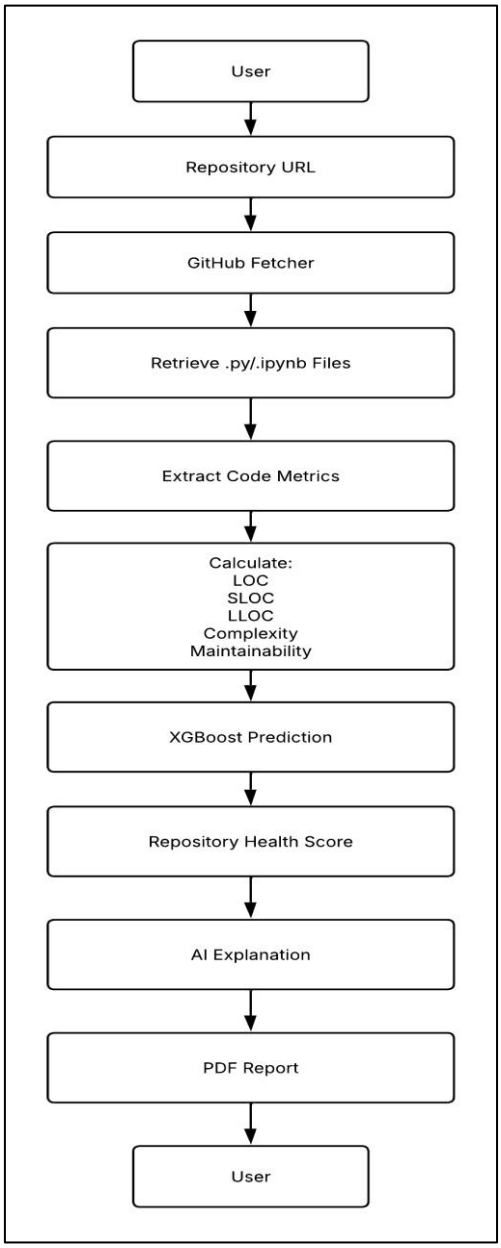


Fig 3.3.3 DFD Level 2

Description

Level 2 DFD provides a detailed view of repository analysis. The GitHub Fetcher retrieves repository files, metrics are extracted, predictions are generated, health scores are calculated, and final reports are delivered to the user.

3.4 Use Case Diagram

The Use Case Diagram describes interactions between the user and the system.

Use Cases

- Analyze Python Code
- Analyze GitHub Repository
- View Metrics
- View Quality Prediction
- View Health Score
- Generate AI Review
- Download PDF Report

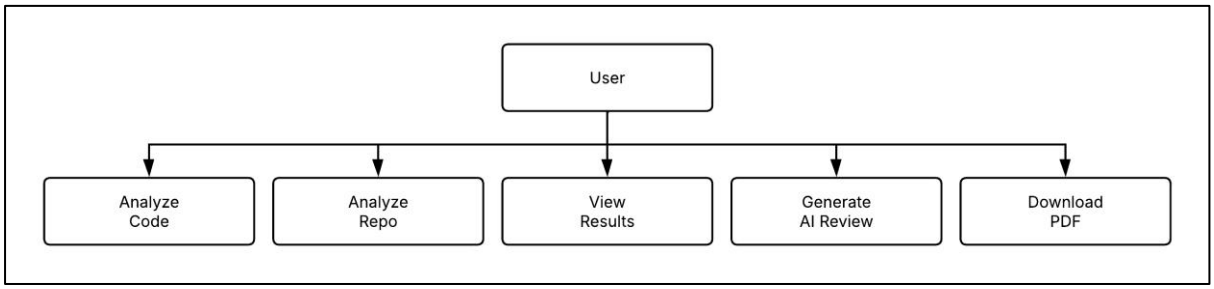


Fig 3.4 Use Case Diagram

Explanation

The user interacts with the system by submitting code or repository URLs. The system processes the input and provides predictions, metrics, AI reviews, and reports.

3.5 Activity Diagram

The Activity Diagram illustrates the workflow of the application.

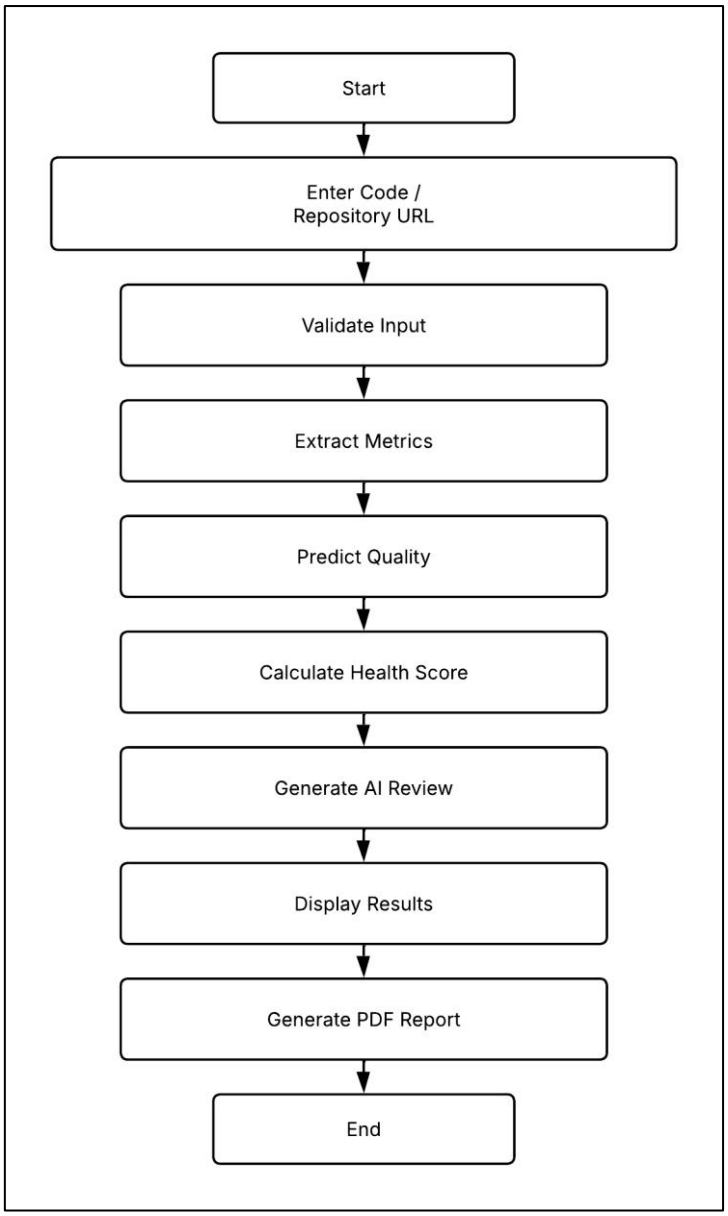


Fig 3.5 Activity Diagram

Explanation

The process begins when the user enters code or a repository URL. The system extracts metrics, predicts code quality, calculates repository health, generates AI explanations, displays results, and optionally creates a PDF report.

3.6 ER Diagram

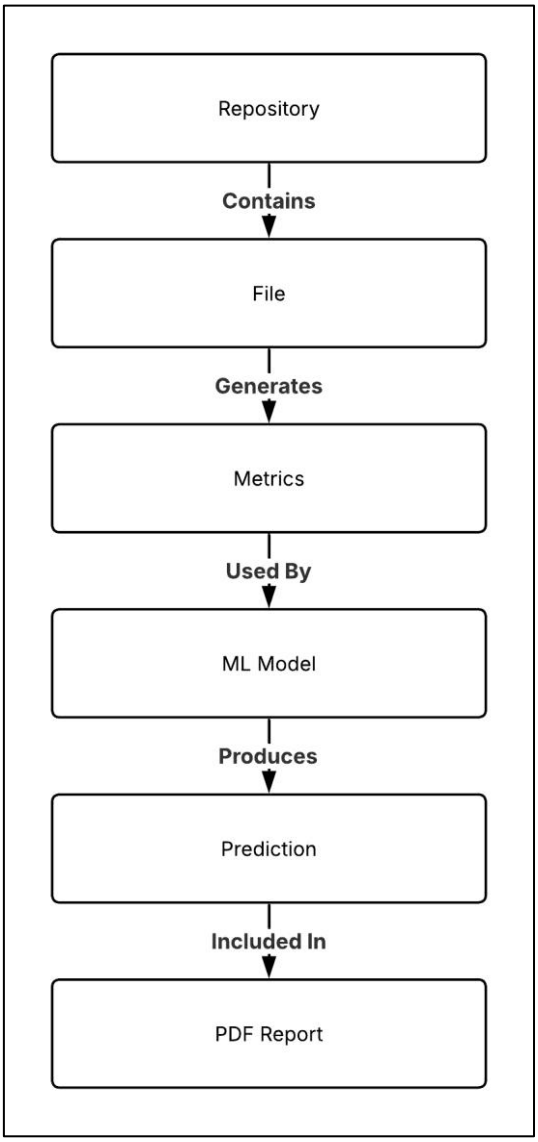


Fig 3.6 ER Diagram

3.7 Design Consideration

Several design principles were considered during development:

Modularity

Each module performs a specific task, making the system easier to maintain and extend.

Scalability

The architecture supports future enhancements such as multi-language analysis and security scanning.

Reusability

Feature extraction, GitHub integration, and report generation modules can be reused independently.

User Friendliness

The Streamlit interface provides a simple and interactive user experience.

Performance

The system efficiently processes source code and repositories while maintaining acceptable response times.

3.8 Summary

This chapter presented the complete system design of the AI Code Review & Quality Analyzer. The architecture, data flow diagrams, use case diagram, activity diagram, and conceptual ER diagram illustrate how different modules interact to perform code quality analysis. The modular design ensures maintainability, scalability, and ease of future enhancements. The next chapter describes the implementation details, modules, algorithms, and technologies used in developing the system.

CHAPTER 4: IMPLEMENTATION

4.1 Introduction

Implementation is the phase in which the system design is converted into a working software application. It involves developing modules, integrating various technologies, testing functionalities, and ensuring that the final system meets the specified requirements.

The AI Code Review & Quality Analyzer has been implemented using Python and Streamlit. The system combines software metric extraction, Machine Learning-based quality prediction, GitHub repository analysis, AI-powered review generation, repository health scoring, visualization, and PDF report generation within a unified platform.

The implementation follows a modular architecture, where each module performs a specific task and communicates with other modules to produce the final analysis results.

4.2 Developed Environment

The project was developed using modern open-source technologies and libraries.
Programming Language

Python

Python was selected because of its simplicity, extensive library support, and strong ecosystem for Machine Learning and Artificial Intelligence applications.

Framework

Streamlit

Streamlit is used to build the web-based user interface. It enables rapid development of interactive applications with minimal code.

4.3 System Module

The application is divided into multiple functional modules.

4.3.1 Code Analysis Module

Purpose

This module allows users to analyze individual Python source code snippets.

Functionality

- Accepts Python code input.
- Performs feature extraction.

- Predicts code quality.
- Generates suggestions.
- Displays visualizations.

4.3.2 Feature Extraction Module

Purpose

The Feature Extraction Module extracts software quality metrics from source code.

Metric	Description
LOC	Total Lines of Code
SLOC	Source Lines of Code
LLOC	Logical Lines of Code
Complexity	Cyclomatic Complexity
Maintainability	Maintainability Index
Functions	Number of Functions
Classes	Number of Classes
Comments	Number of Comments

Importance

These metrics serve as input features for the Machine Learning model.

4.3.2 Machine Learning Prediction Module

Purpose

This module predicts code quality using a trained XGBoost model.

Model Used

XGBoost Classifier

Quality Classes

Label	Meaning
Good	High Quality Code
Average	Moderate Quality
Poor	Low Quality Code

Benefits

- Fast prediction
- High accuracy
- Handles multiple metrics efficiently

4.3.4 GitHub Repository Analysis Module

Purpose

This module analyzes public GitHub repositories.

Features

- Repository URL parsing
- Python file retrieval
- Jupyter Notebook support
- File-wise analysis

Supported Files

- .py files
- .ipynb files

Advantages

- Automated repository analysis
- Supports large repositories
- Provides repository-level insights

4.3.5 File Level Analysis

Purpose

To evaluate each repository file independently.

File Name	Quality	Maintainability	Complexity
File1.py	Average	85	3
File2.py	Good	72	7

Benefits

- Identifies problematic files

- Enables targeted improvements
- Improves maintainability

4.3.6 Repository Health Score

Purpose

To calculate an overall quality score for the repository.

Formula

The Repository Health Score is computed using:

- Maintainability Index
- Cyclomatic Complexity
- Documentation Quality
- Code Structure

Example

Score =82/100

4.3.7 AI Review Module

Purpose

To generate human-readable explanations and recommendations.

Technology

Large Language Model (LLM)

Inputs

- Source code
- Prediction result
- Extracted metrics ex

Outputs

- Strengths of the code
- Weaknesses of the code
- Improvement recommendations

Example

The code demonstrates good modularity and maintainability. Consider reducing complexity in nested functions to improve readability.

4.3.8 Visualization Module

Purpose

To present analysis results graphically.

Charts Used

- Bar Charts
- Metric Comparisons
- Repository Statistics

Library

Plotly Express

Benefits

- Easy interpretation
- Interactive analysis
- Better user experience

4.3.9 PDF Report Generation Module

Purpose

To generate professional reports.

Contents of Report

- Repository Details
- Quality Prediction
- Health Score
- Metrics Summary
- File-Level Analysis
- AI Review

Library

ReportLab

Benefits

- Easy sharing
- Documentation
- Professional presentation

4.4 Summary

This chapter described the implementation details of the AI Code Review & Quality Analyzer. It explained the technologies used, system modules, algorithms, and important code snippets. The modular implementation enables efficient code analysis, repository evaluation, health score calculation, AI-powered review generation, and report creation. The next chapter presents the testing methodology, experimental results, screenshots, and discussion of the system's performance.

CHAPTER 5: TESTING AND RESULTS

5.1 Introduction

Testing is a crucial phase of software development that ensures the developed system functions correctly, efficiently, and reliably. The purpose of testing is to identify defects, validate requirements, and verify that the system performs as expected under different conditions.

The AI Code Review & Quality Analyzer was tested using various Python code samples and public GitHub repositories. Different testing techniques were applied to evaluate the correctness of feature extraction, machine learning prediction, repository analysis, health score generation, AI review generation, and PDF report creation.

This chapter presents the testing methodology, test cases, screenshots, results, and performance analysis of the proposed system.

5.2 Testing Strategy

The following testing approaches were used during development.

5.2.1 Unit Testing

Unit testing verifies individual modules independently.

Modules Tested

- Feature Extraction Module
- GitHub Fetcher Module
- Machine Learning Prediction Module
- Health Score Module
- PDF Report Module

Objective

To ensure each module performs its intended functionality correctly.

5.2.2 Integration Testing

Integration testing validates interactions between multiple modules.

Examples

- GitHub Fetcher + Feature Extraction
- Feature Extraction + XGBoost Model
- Prediction Module + AI Review Module

- Analysis Module + PDF Report Generator

Objective

To verify smooth communication between modules.

5.2.3 Functional Testing

Functional testing ensures that all user requirements are satisfied.

Functions Tested

- Code Analysis
- Repository Analysis
- File-Level Analysis
- Repository Health Score
- AI Review Generation
- PDF Report Download

5.2.4 User Interface Testing

The Streamlit interface was tested to verify:

- Input handling
- Button functionality
- Result display
- Visualization rendering
- Report download functionality.

5.3 Screenshots

5.3.1 Home page

Description:

The home page allows users to choose between Code Analysis and GitHub Repository Analysis.

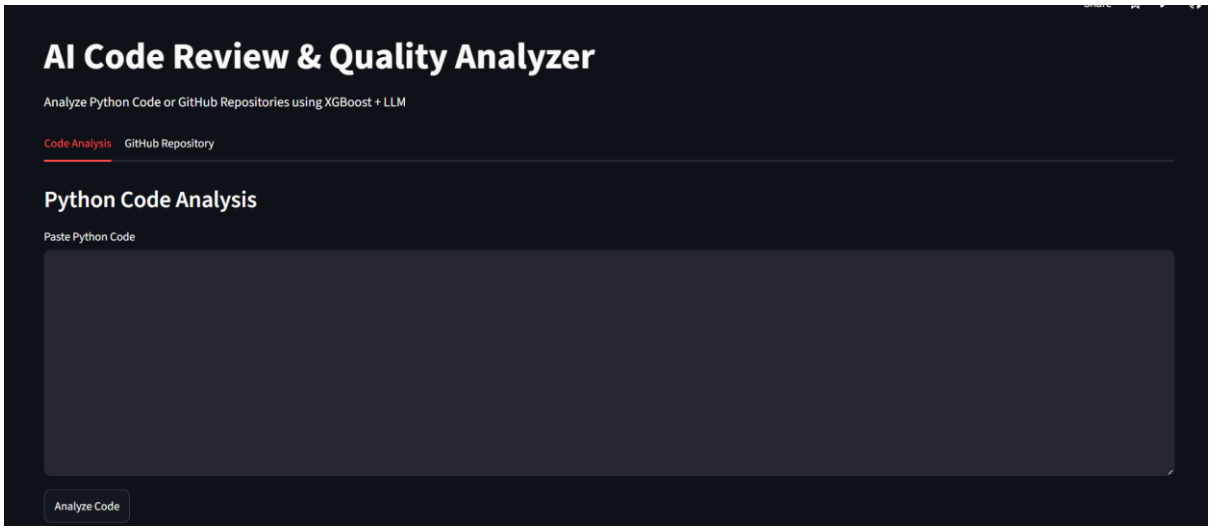


Fig 5.3.1 Home Page

5.3.2 Quality Prediction Result

Description:

The system predicts code quality as Good, Average, or Poor.

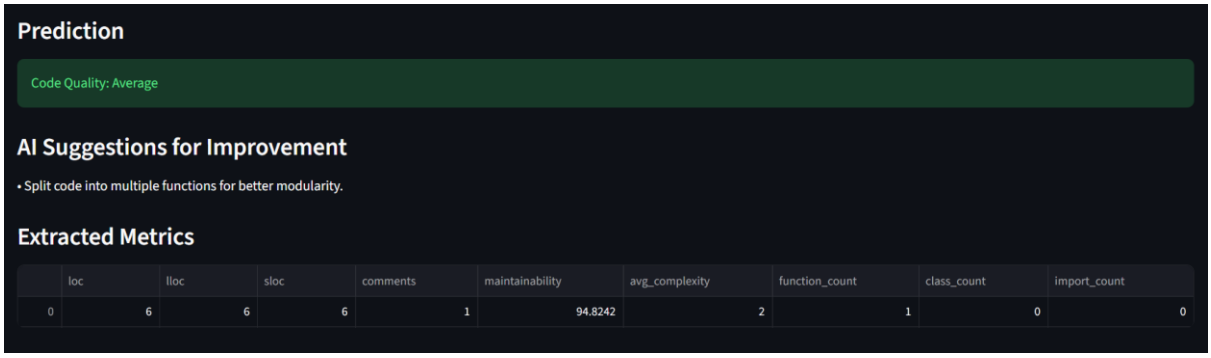


Fig 5.3.2 Quality Prediction Result

5.3.3 Metric Visualization

Description:

Bar charts display extracted software metrics.

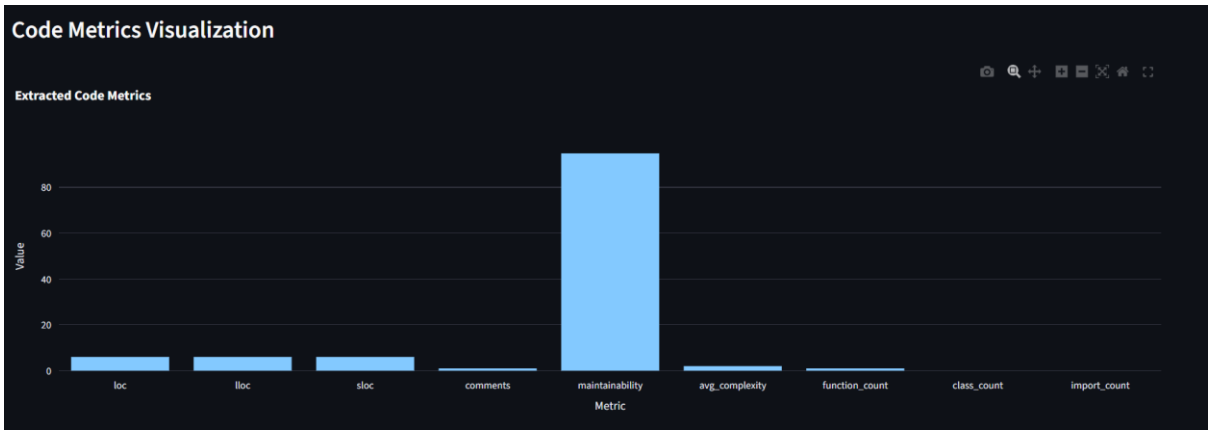


Fig 5.2.3 Metric Visualization

5.3.4 GitHub Repository Analysis

Description:

The user enters a repository URL for analysis.

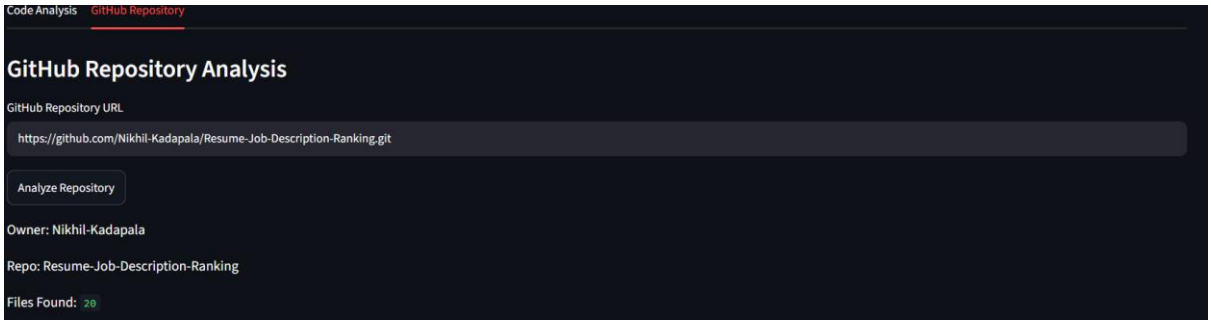
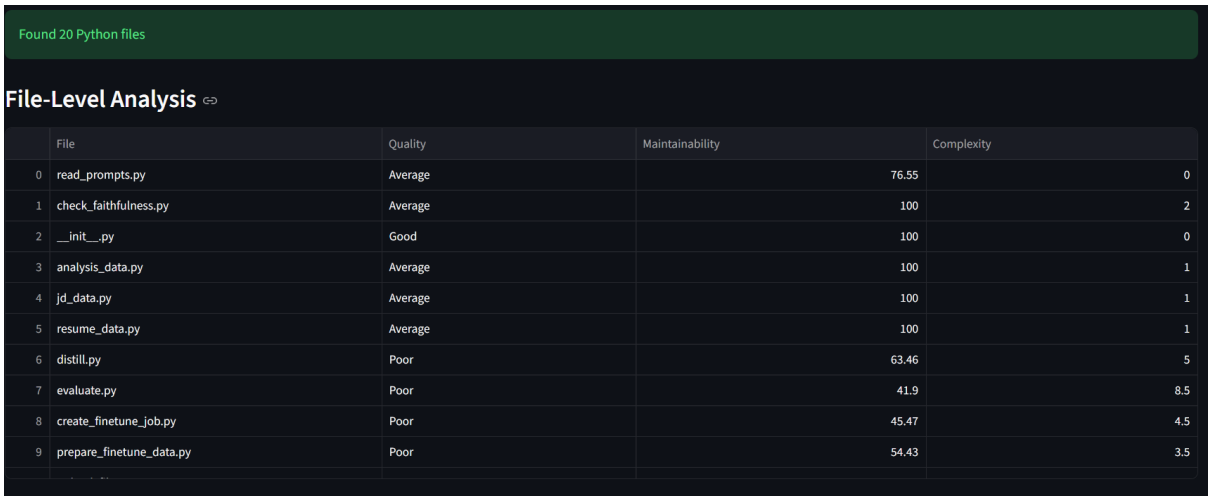


Fig 5.3.4 GitHub Repository Analysis

5.3.5 File Level Analysis

Description:

Individual files are analyzed separately.



	File	Quality	Maintainability	Complexity
0	read_prompts.py	Average	76.55	0
1	check_faithfulness.py	Average	100	2
2	__init__.py	Good	100	0
3	analysis_data.py	Average	100	1
4	jd_data.py	Average	100	1
5	resume_data.py	Average	100	1
6	distill.py	Poor	63.46	5
7	evaluate.py	Poor	41.9	8.5
8	create_finetune_job.py	Poor	45.47	4.5
9	prepare_finetune_data.py	Poor	54.43	3.5

Fig 5.3.5 File Level Analysis

5.3.6 AI Repository Review

Description:

The system generates AI-powered explanations and recommendations.

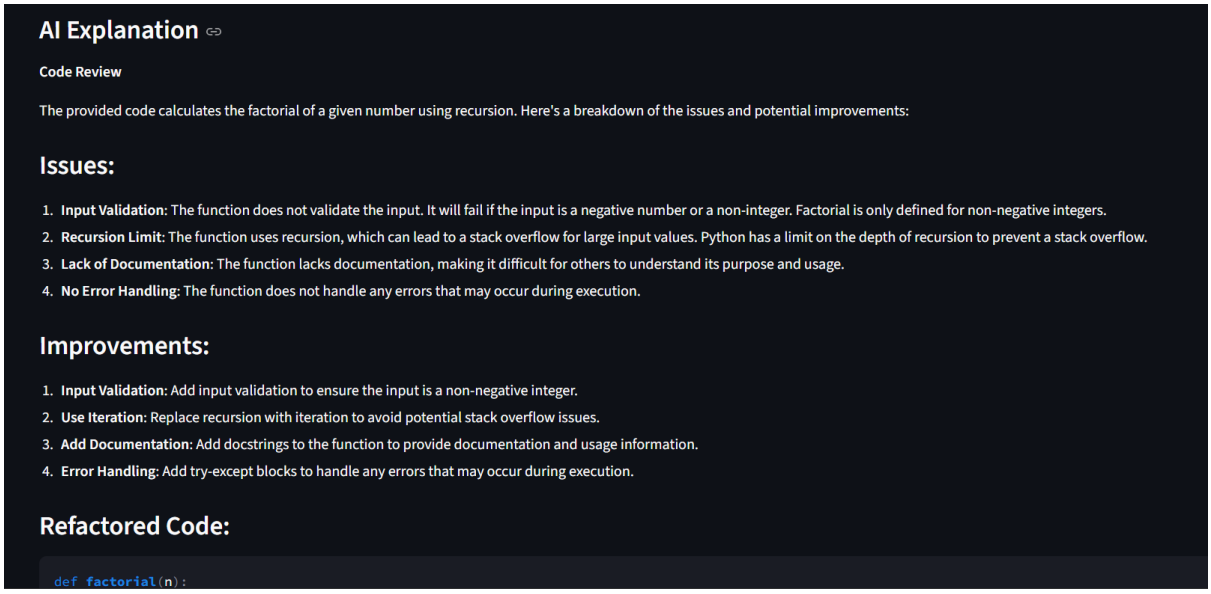


Fig 5.3.6 AI Repository Review

5.4 Discussion

The results demonstrate that the proposed system effectively automates code quality assessment and repository analysis.

Observations

Feature Extraction

The system successfully extracted software engineering metrics from both Python files and Jupyter Notebooks.

Quality Prediction

The XGBoost model effectively classified source code into quality categories.

Repository Analysis

The GitHub integration enabled automatic retrieval and analysis of repository contents.

Health Score

The Repository Health Score provided a quick and intuitive representation of repository quality.

AI Recommendations

The AI-generated reviews helped users understand strengths and weaknesses within their codebase.

Visualization

Interactive charts improved the readability and interpretation of software metrics.

5.5 Summary

This chapter presented the testing methodology and experimental results of the AI Code Review & Quality Analyzer. Various testing techniques were used to validate system functionality, performance, and reliability. The results demonstrated that the system successfully analyzes Python code and GitHub repositories, predicts quality, calculates repository health scores, generates AI-powered recommendations, and creates professional PDF reports. The next chapter presents the conclusion, achievements, and future enhancements of the project.

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

Software quality plays a vital role in determining the reliability, maintainability, and performance of software systems. As software projects continue to grow in complexity, traditional manual code review processes become increasingly time-consuming, inconsistent, and difficult to scale. Developers require automated tools that can quickly assess code quality and provide meaningful recommendations for improvement.

The AI Code Review & Quality Analyzer was developed to address these challenges by combining software metric extraction, Machine Learning-based prediction, GitHub repository analysis, repository health scoring, AI-powered review generation, and PDF reporting within a single platform.

The system successfully extracts important software engineering metrics such as:

- Lines of Code (LOC)
- Source Lines of Code (SLOC)
- Logical Lines of Code (LLOC)
- Cyclomatic Complexity
- Maintainability Index
- Function Count
- Class Count
- Comment Count

These metrics are used as input to a trained XGBoost Machine Learning model that predicts the quality of source code as Good, Average, or Poor. The integration of Machine Learning enables objective and consistent evaluation of software quality.

One of the major achievements of the project is the implementation of GitHub Repository Analysis. The system automatically retrieves Python (.py) files and Jupyter Notebook (.ipynb) files from public repositories and performs detailed analysis on each file. This capability significantly reduces the effort required to evaluate large repositories manually.

The File-Level Analysis feature helps developers identify problematic files and prioritize improvement efforts. Additionally, the Repository Health Score provides a simple yet effective method for summarizing the overall condition of a repository.

The integration of a Large Language Model (LLM) further enhances the usefulness of the system by generating human-readable explanations and recommendations. Instead of merely presenting numerical metrics, the system helps users understand the implications of the analysis results and suggests possible improvements.

The Streamlit-based user interface offers a simple and interactive environment for performing code reviews, repository analysis, and report generation. The PDF reporting functionality enables users to save and share analysis results in a professional format.

Overall, the project successfully achieved all its objectives and demonstrated the practical application of Artificial Intelligence and Machine Learning in software engineering. The developed system improves developer productivity, reduces manual review effort, and promotes better coding practices through automated quality assessment.

6.2 Future Scope

Although the current implementation successfully achieves its objectives, several enhancements can further improve the capabilities and usefulness of the system.

6.2.1 Multi-Language Support

The current version focuses on Python code analysis. Future versions can support additional programming languages such as:

- Java
- C++
- JavaScript
- TypeScript
- C#
- Go

This enhancement would increase the applicability of the system across diverse software projects.

6.2.2 Security Vulnerability Analysis

Future versions can integrate security analysis tools such as:

- Bandit
- Semgrep

- CodeQL

This would allow the system to detect security vulnerabilities in addition to code quality issues.

6.2.3 Pull Request Analysis

Integration with GitHub Pull Requests could enable automatic review of code changes before merging.

Benefits include:

- Continuous quality monitoring
- Automated feedback
- Faster development cycles

6.2.4 CI/CD Integration

Future versions can integrate with Continuous Integration and Continuous Deployment (CI/CD) pipelines.

Examples include:

- GitHub Actions
- Jenkins
- GitLab CI/CD
- Azure DevOps

This would allow automatic code quality checks during software development workflows.

6.2.5 Team Dashboard

A centralized dashboard can be developed to monitor:

- Team code quality
- Repository trends
- Health score history
- Developer performance metrics

This feature would be particularly useful for software development organizations.

6.2.6 Historical Repository Analysis

Future versions can track repository quality over time and provide trend analysis.

Features may include:

- Quality growth charts
- Complexity evolution
- Maintainability tracking
- Technical debt monitoring

CHAPTER 7: REFERENCES

- Python Documentation. Python Software Foundation. Available at.

  https://docs.python.org

- **Streamlit Documentation.** Available at: https://docs.streamlit.io
- **XGBoost Documentation.** Available at: https://xgboost.readthedocs.io
- **Scikit-Learn Documentation.** Available at: https://scikit-learn.org
- **Plotly Documentation.** Available at: https://plotly.com
- **GitHub REST API Documentation.** Available at: https://docs.github.com
- **ReportLab Documentation.** Available at: https://www.reportlab.com
- Clean Code, Prentice Hall, 2008.
- Code Complete, Microsoft Press, 2nd Edition, 2004.
- Research papers on Software Quality Prediction, Machine Learning in Software Engineering, and AI-Assisted Code Review from scholarly journals and conference proceedings.